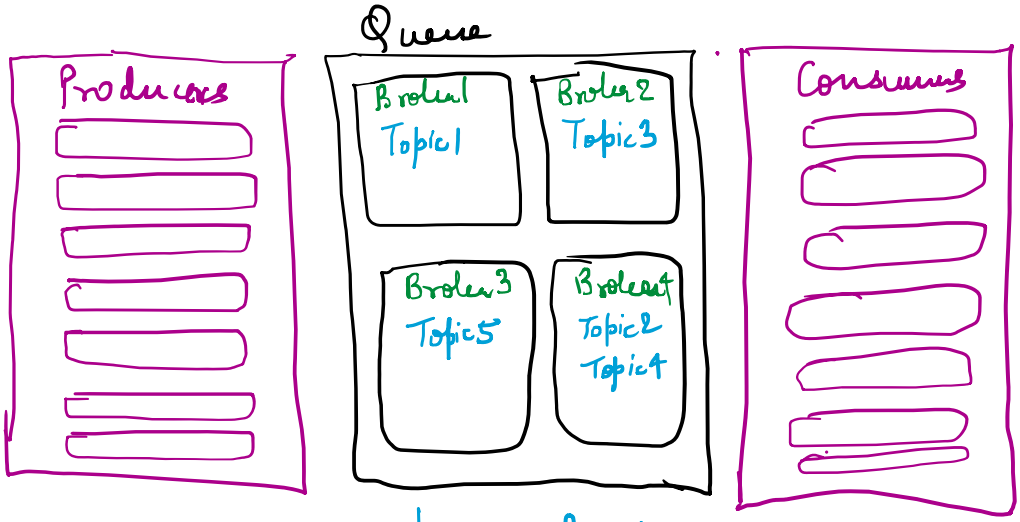
... Contd.

Kafka

KAFKA

↳ Implementⁿ of Persistent Queues.

How KAFKA works



↳ throughput of millions of events/sec.

→ Kafka internally works across multiple machines

↳ Kafka Brokers.

Problems if every topic assigned to only 1 m/c:—

1. If broker is down?

Problems if every topic assigned to one broker

↳ What if broker is down?

SPOF

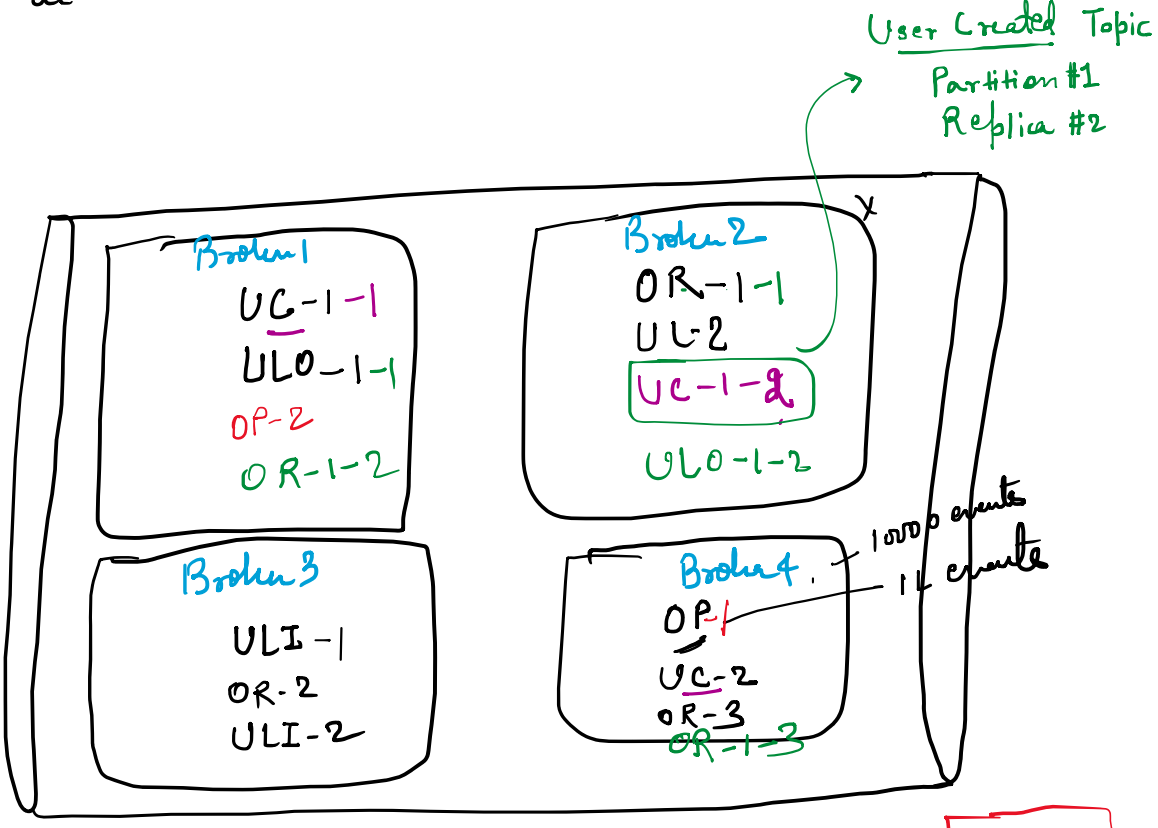
↳ events of that topic lost

Sol^{n} :- Replicas

↳ What if 1 m/c can't fit all events of a topic

Divide a topic across brokers

Sol^{n} → partitioning.



Each Topic — divided across brokers → Partitions

How to divide data b/w brokers?

Topics	Brokers
UC User Created	
OP Order Placed	
OR Order Refund	

ULI User login

ULO User logout

What is Sharding to DB
Partitions to Broker. Topic.
=> Sharding to Sharding key
Similarly partition to partitioning key

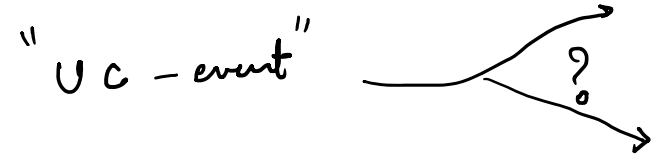
Q Can 2 partitions of same topic be in same broker?

Ans. YES

Q Can two replicas of same partition (of same Topic) be in same broker?

Ans. NO

Q



How its decided that a event will go to which partition?

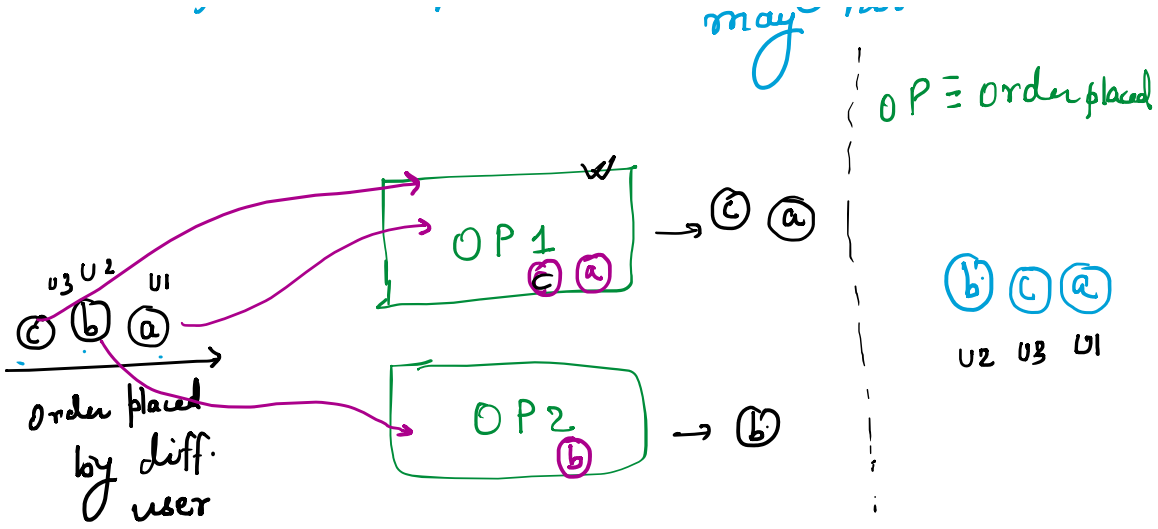
↳ Consistent Hashing

How replicas will be handled?

↳ Master-Slave Archi.

- Producers
- Consumers
- MQ / Kafka
- Brokers
- Topics
- Partitions
- Replicas

- Queue (FIFO)
  - ↳ within a partition, events are FIFO
  - ↳ across partitions → may be FIFO, may not be FIFO.



Prob.

What if these are orders placed by same user (diff. events from user going to diff. partitions)

Sol^{n}

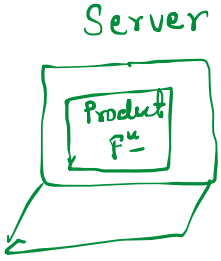
All events from the same user go to the same partition.

user-id is the partition key.

Microservices

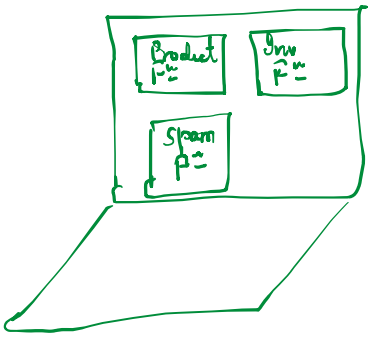
Shipkart.com (2008)

100 users/day ⟹ 10 orders/day.



2012

10k users/day ⟹ 1k orders/day

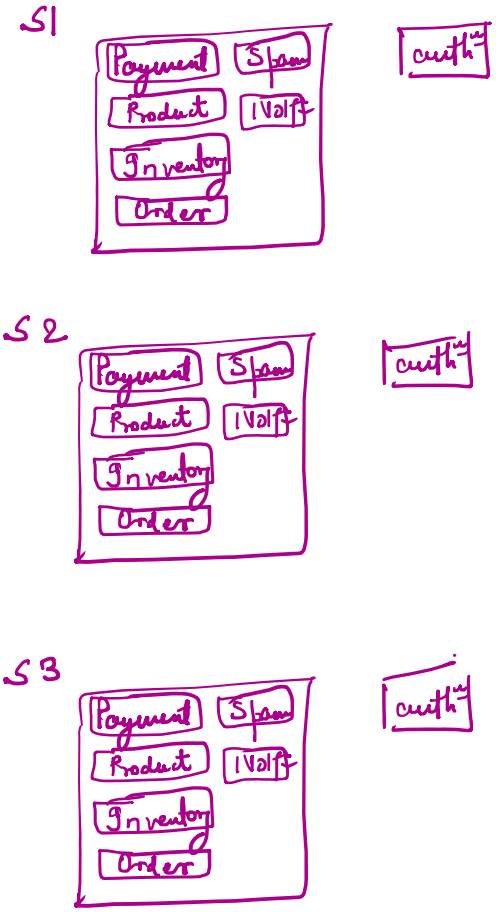


2014

10k ⟶ 1M users/day

10k → 1M users/day

1k → 100k orders/day

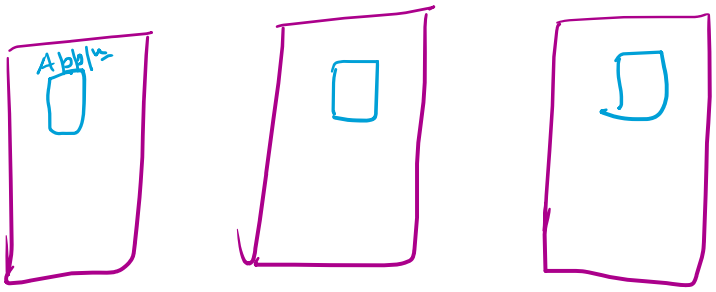


src/
com/
shiphart/
product/
payment/
authentication/

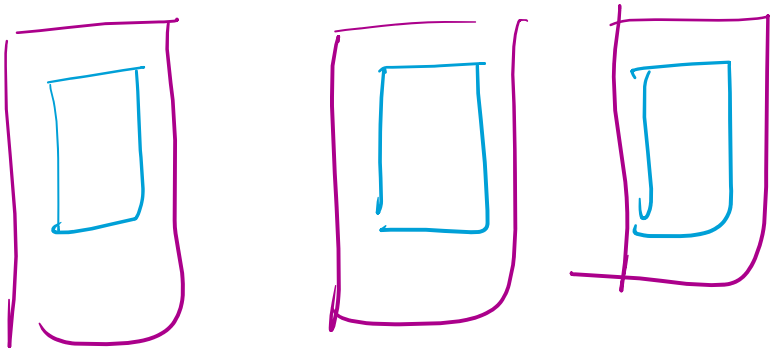
notfn

1. As a company grows, the size of codebase also keeps incing

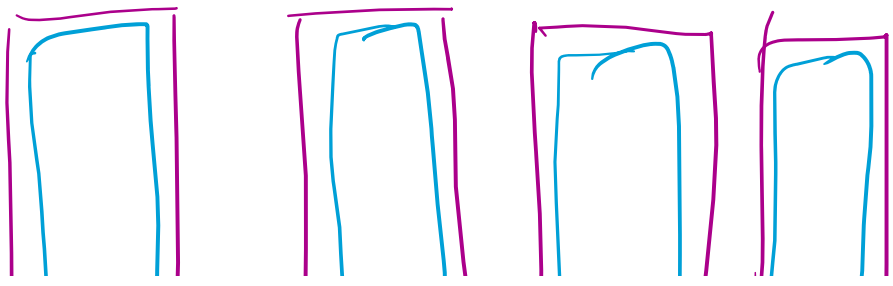
2014



2018

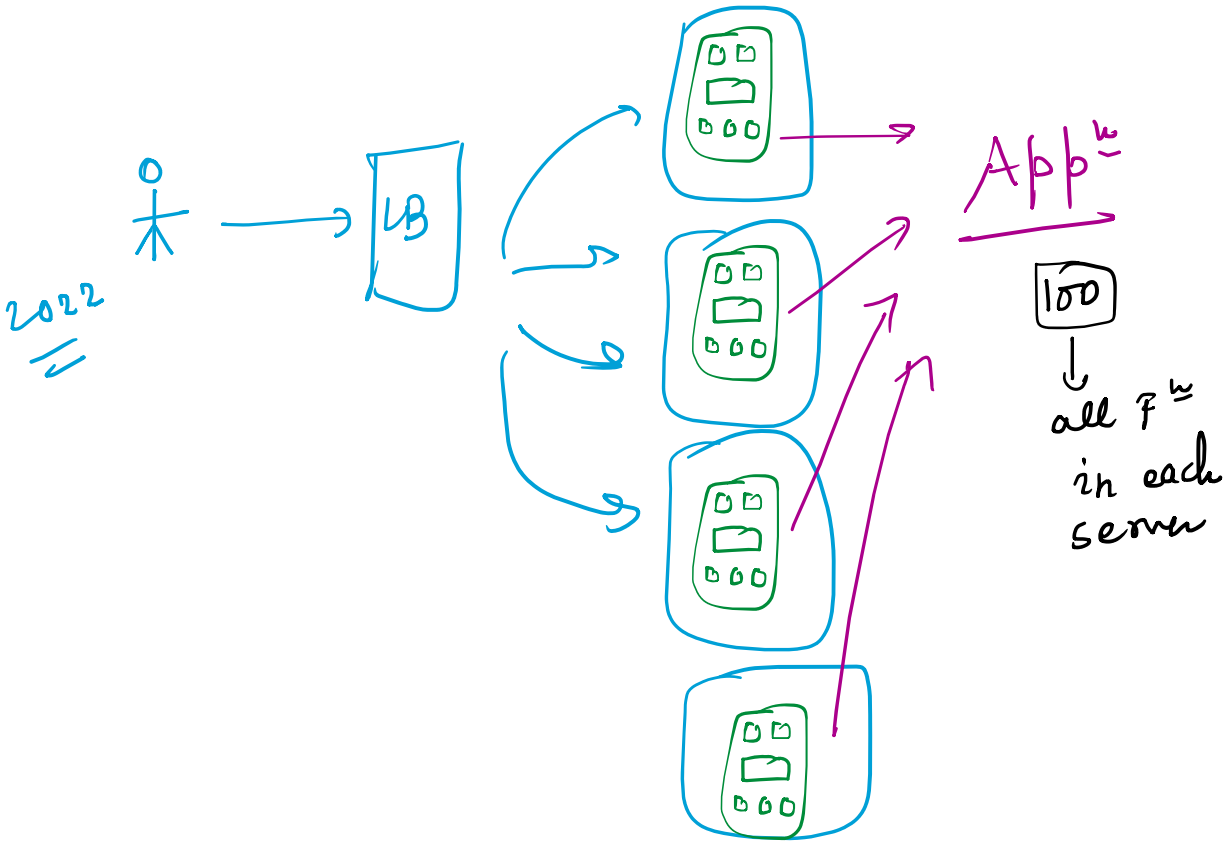


2022



2022

1. Huge codebase
2. Multiple f^{n} w/in codebase
3. Running on multiple servers



2022

Cons of a Huge Codebase

1.) Huge time to compile
   ↳ Productivity of devs ↓

2. ... in 1 part of codebase

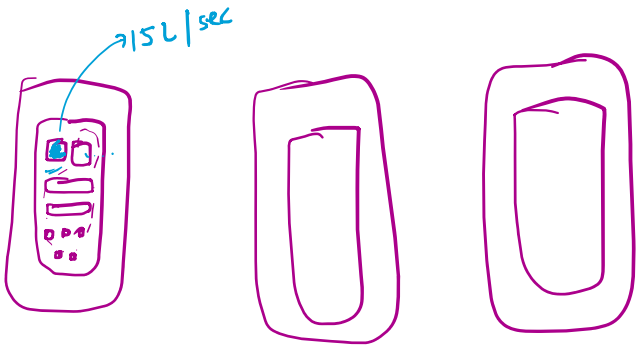
2.) An issue in 1 part of codebase may end up bringing other (all) sys. down.

3.) Tight coupling

4.) Legacy Tech Stack
↳ current design decisions are tied to past decisions

5.) No selective Scaling ⇒ unnecessary res. uses.

6.) Dev onboarding time ↑



(A) Pros.

1.) Sharing knowledge is easy.
↳ no overload of cross-collaboration

↳ cross-collaboration

↳ Devops work ↓

2.) Talking to another functionality is as simple as a f^n call

↳ N/W overhead ↓

↳ overall latency ↓↓

~~3.~~) [Sometimes] Cost ↓↓

4.) Testing everything easily

Monolithic Appⁿ

S/W sys. where the complete codebase is part of a single applⁿ.

As long as, entire applⁿ goes in as a single build package

its = Monolithic archi.

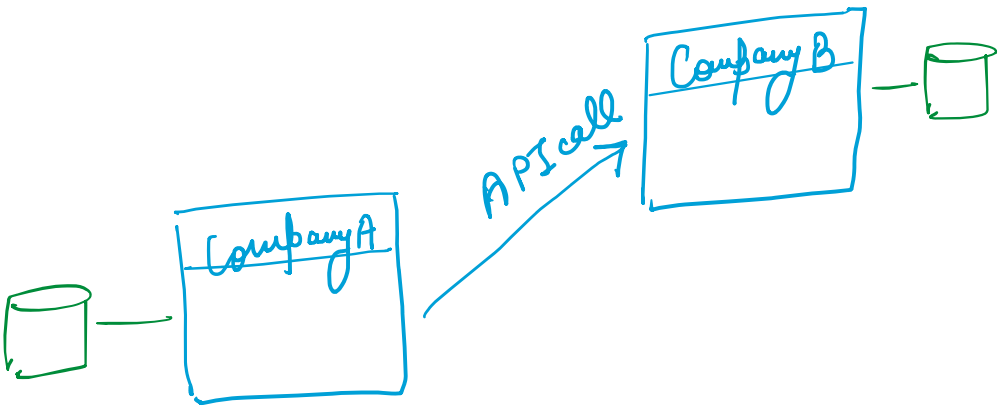
Single ⟶ Multiple

Single → Multiple

Microservices

When the S/w sys. of a company is divided into more than one independent app^n

An ideal microservice works like an independent company.



Pros → Cons. of Monolithic

Cons → Pros " "

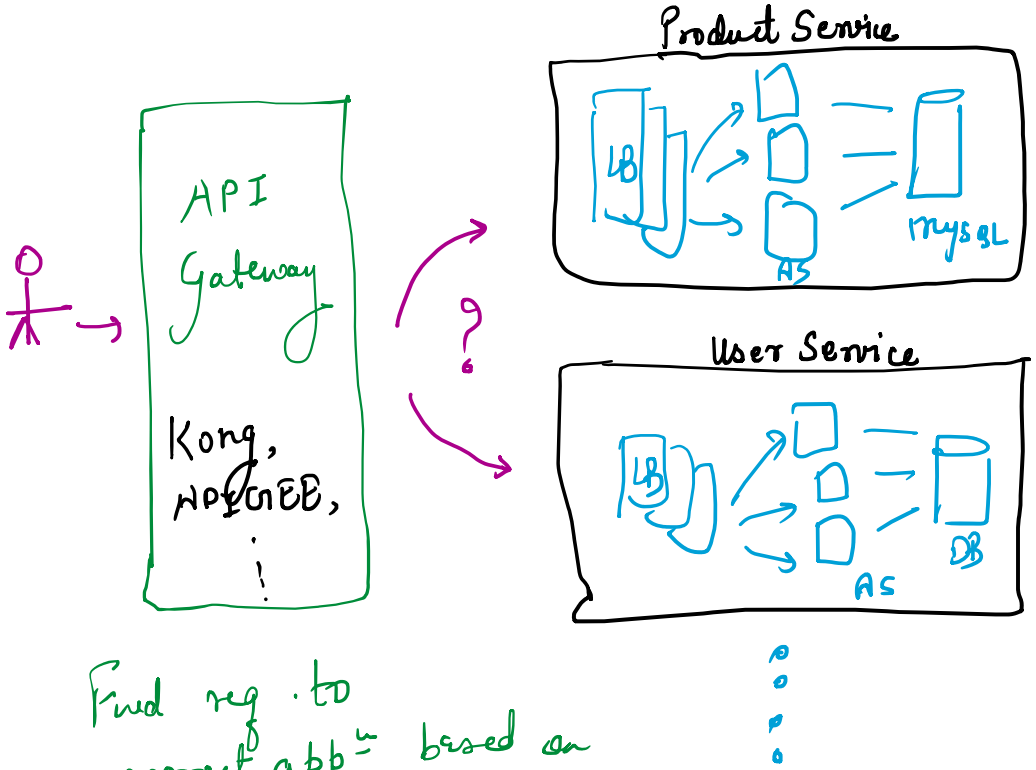
API Gateway

product service

API1 www.flipkart.com/products/101.

API2 " " " /users/1029

↳ user service.



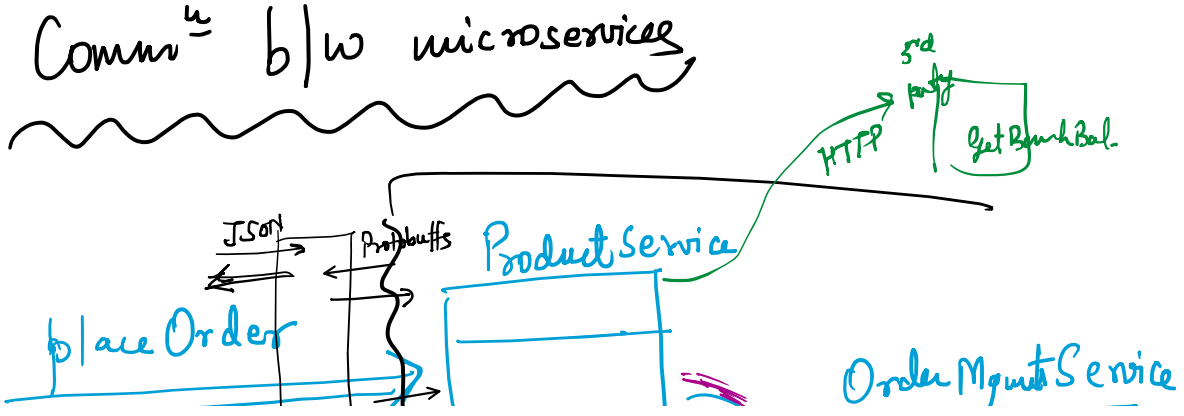
Find req. to correct app^{n} based on

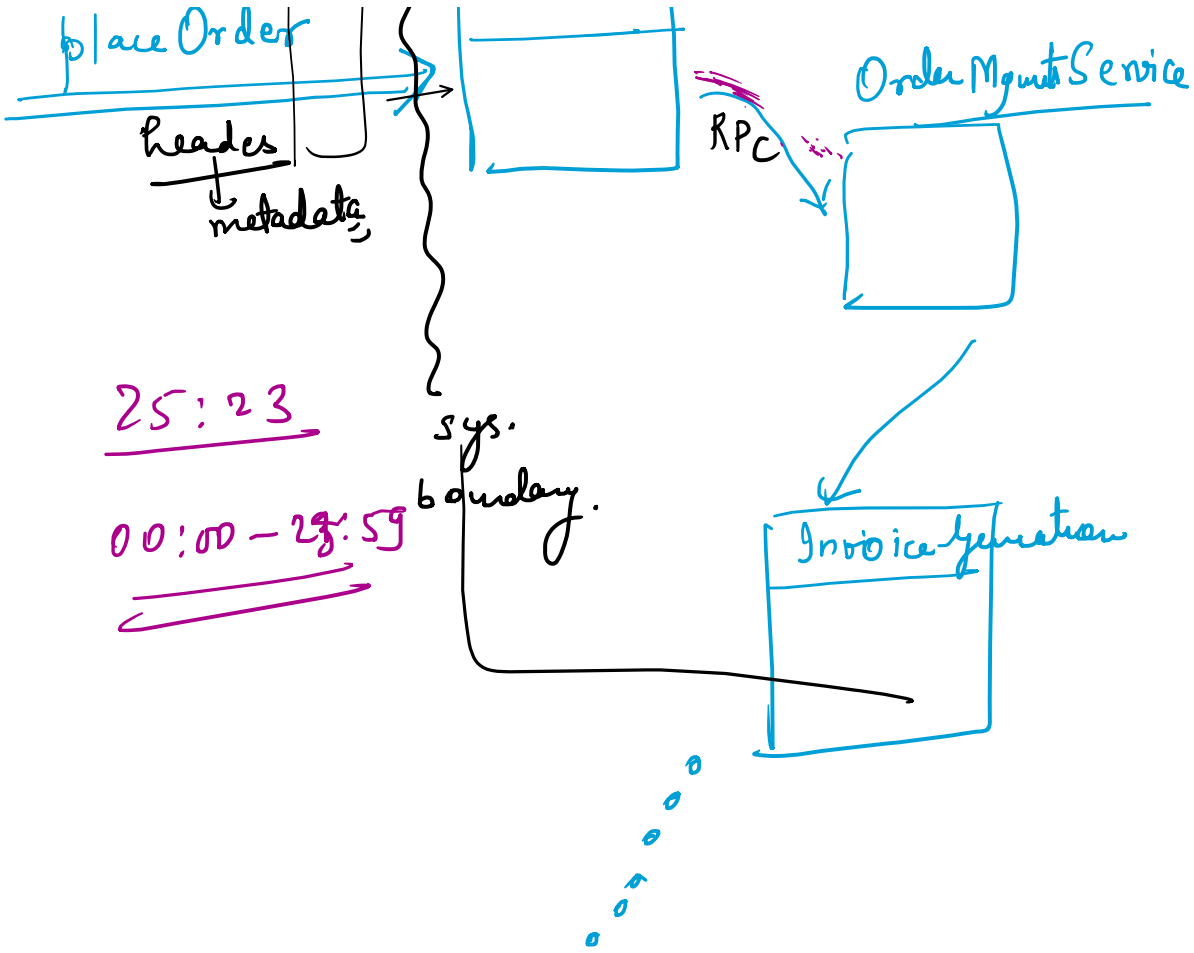
1.) path

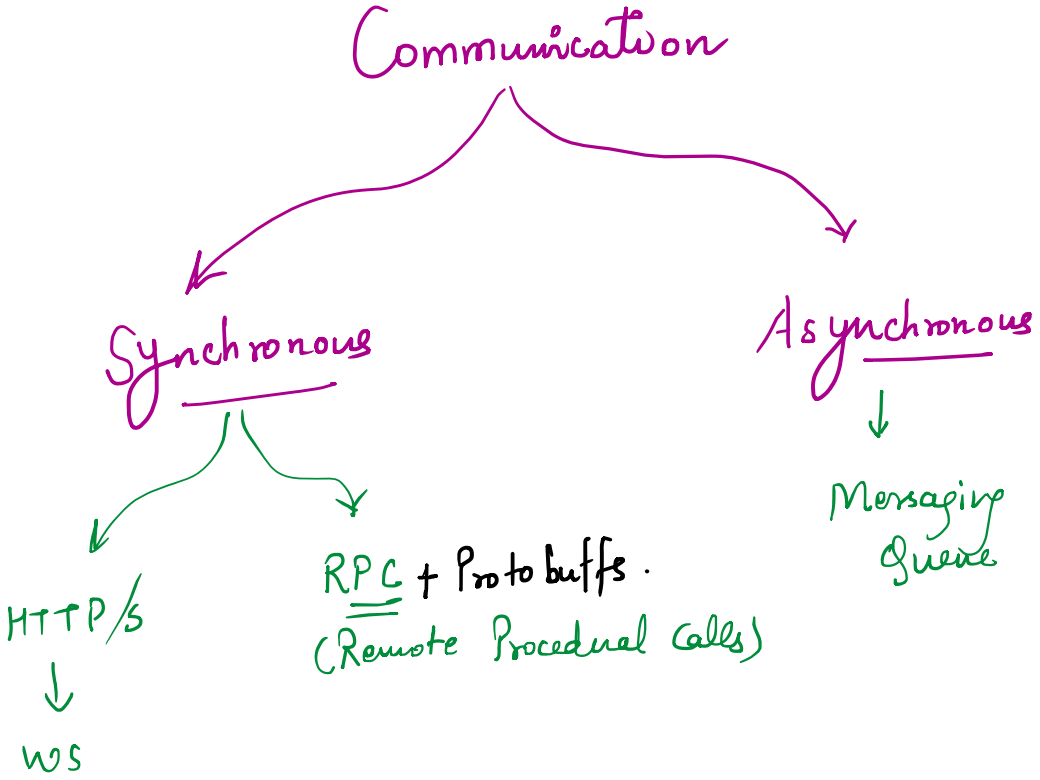
2.) req Body

3.) Headers

⊛ Comm^{n} b/w microservices




Place Order
Headers
metadata
RPC
Order Mgmt Service
25:23
00:00 - 23:59
sys.
boundary.
Invoice Generation


Communication
Synchronous
Asynchronous
HTTP/S
WS
RPC + Protobuffs.
(Remote Procedural Calls)
Messaging Queue

Sync → HTTP/S →

https://www.....

headers .

↓

typically 1kB

RPC

↳ Companies break HTTP req.

@ API G/W

into RPC

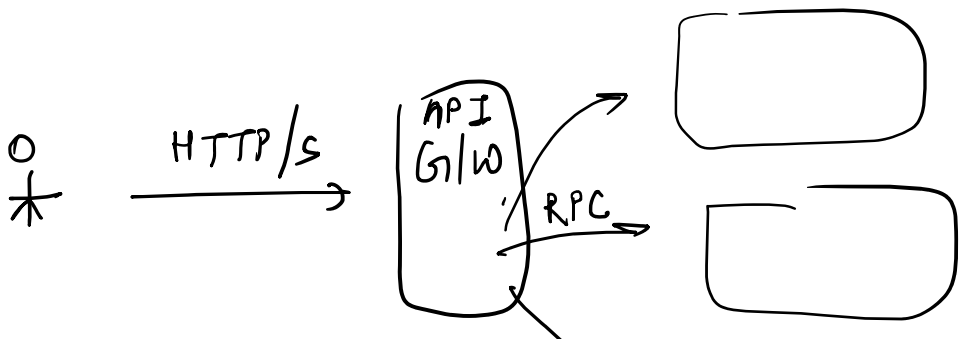
(remove headers)

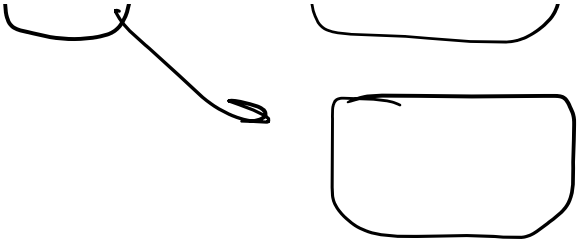
internal comm^n happen via RPC

↳ No headers

↳ No overhead

↳ Overall latency ↓↓

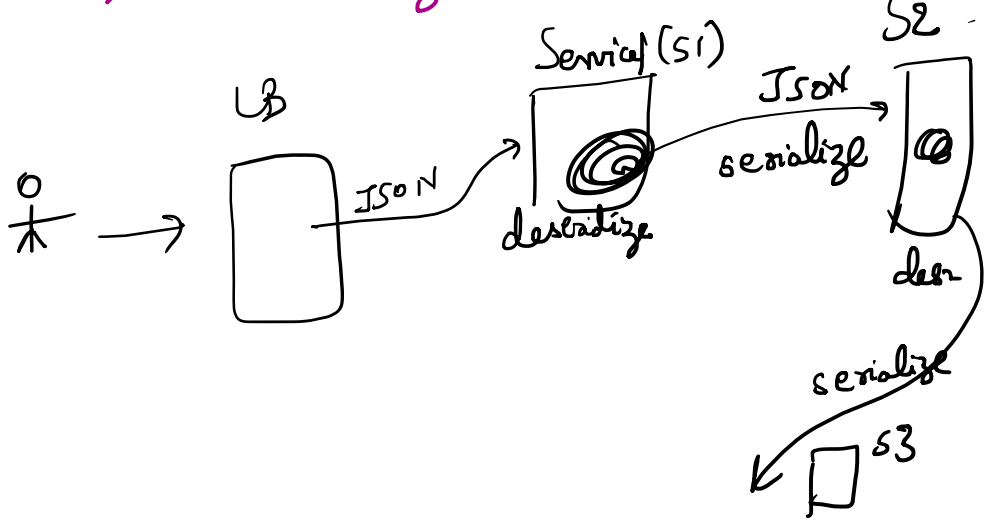




Addl overhead (in https)

↳ i.) Headers ✓

↳ ii) Serialize/Deserialize to JSON

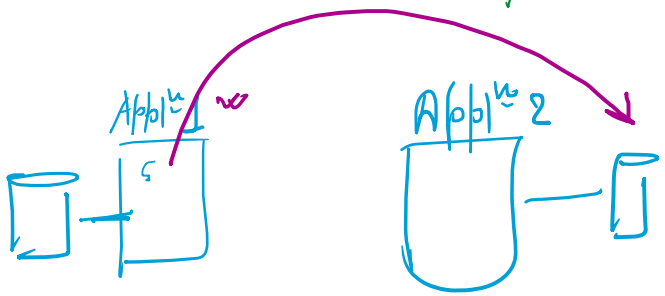


∴ RPC uses Protobuffs (instead of JSON)

↳ Bytes[] that can be read by any prog. lang.

(*) SOA → Service Oriented

(*) SOA → Service Oriented Archi-



If Appl^n 1 has to access data of appl^n 2 → it can directly make a call to the DB

not allowed in typical Microservice (use API-call)